

ICS2121 Métodos de Optimización.

Notas breves de Programación Entera

Prof. Jorge R. Vera
Dept. de Ingeniería Industrial y de Sistemas
Escuela de Ingeniería, Pontificia Universidad Católica de Chile

Noviembre 2025

1 Introducción

En el curso hemos estado preocupados de abordar problemas estructurados y muy grandes, los cuales aparecen en muy diversas aplicaciones. Si bien estas técnicas nos han permitido reducir en algo la complejidad, hemos notado que muchas veces terminamos con un problema maestro o con un satélite, que corresponde a un problema de Programación Entera (o enteros mixtos). Poder resolver eficientemente, entonces, esos problemas es muy importante.

La Programación Entera por sí misma es una parte muy importante de la Optimización y ha sido abordada con profundidad, existiendo muchos conceptos. En estas notas, correspondientes a la parte de curso sobre el tema, complementaremos lo visto en clases. El tema lo veremos de manera introductoria pero igual se avanzará más allá de lo básico que se estudia en el curso de Optimización.

Partiremos revisando el algoritmo más básico para Programación Entera (PE), que es el Algoritmo de Ramificación y Acotamiento. Después discutiremos la geometría subyacente de la programación entera, la noción de formulaciones y desigualdades válidas y luego la idea de planos cortantes.

2 El Algoritmo de Ramificación y Acotamiento

Presentaremos en esta sección el primer método que podríamos usar para resolver un problema de Programación Entera. Para esto consideremos (sin pérdida de generalidad) un problema de la siguiente forma:

$$\begin{array}{ll} IP) & z^* = \min \quad c^T x \\ & s.a. \quad Ax \leq b \\ & \quad \quad x \geq 0, \text{ entero} \end{array}$$

Supongamos inicialmente que la región factible es acotada. Como los puntos enteros son discretos, tenemos que hay sólo un número finito de posibles soluciones al problema. Esto lleva de inmediato a la pregunta: ¿por qué no enumerarlas todas y elegir la mejor?

Supongamos, a modo de ilustración, que pudiéramos estimar un número entero K tal que $0 \leq x_j \leq K, \forall j$. El número total de posibles soluciones que podría tener el problema puede ser fácilmente acotado por $(K+1)^n$ (¿Por qué?). Por ejemplo, si $K = 5$ y $n = 3$, entonces habrá a lo más 125 posibles soluciones. Pero si $n = 80$, la cota al número de soluciones es:

$$82.718.061.255.302.767.487.140.869.206.996.285.356.581.211.090.087.890.625$$

o, aproximadamente, $8,2 \times 10^{55}$.

Pregunta: Suponga que su computador puede generar y evaluar un millón de posibles soluciones por segundo. El universo se supone que tiene una edad aproximada de 14.000 millones de años. Si aceptamos

la teoría cosmológica de la expansión y posterior contracción y colapso, estamos más o menos a la mitad de la “vida” del universo. ¿Alcanzará el tiempo para evaluar todas las soluciones posibles del caso anterior? (asumiendo que su computador pueda sobrevivir funcionando a todos nosotros)

Si bien es verdad que no todas esas posibles soluciones obtenidas por enumeración serán válidas, queda claro que enumerar “a tontas y a locas” puede no ser la mejor de las estrategias. Lo que mostraremos ahora es una de las primeras ideas que fue desarrollada para resolver problemas enteros. Está basada en la idea de enumerar, pero permite guiar esta enumeración de una forma más o menos inteligente.

2.1 La Idea de Ramificación y Acotamiento

Retomemos el problema entero IP) de más arriba. La relajación lineal del problema es:

$$P_0) \quad \begin{aligned} z^0 &= \min && c^T x \\ &s.a. && Ax \leq b \\ &&& x \geq 0 \end{aligned}$$

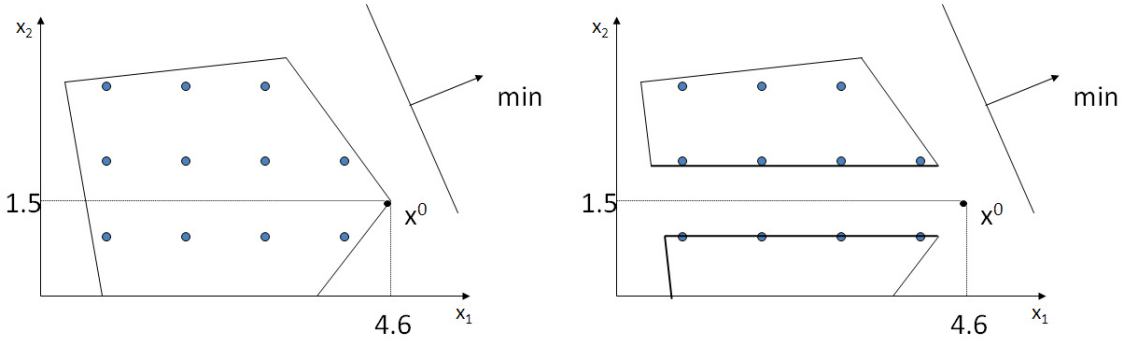
Supongamos que hemos resuelto la relajación lineal de este problema y hemos obtenido x^0 como solución óptima de la relajación lineal. Si todas las componentes de x^0 son enteras, entonces estamos listos: x^0 es solución óptima del problema IP) (¿Por qué?). Si no, sea $k \in \{1, \dots, n\}$ un índice tal que la k -ésima coordenada, x_k^0 , sea fraccionaria. Ahora creamos dos problemas lineales a partir de la relajación lineal y el índice k :

$$\begin{aligned} P_1) \quad z^1 &= \min && c^T x \\ &s.a. && Ax \leq b \\ &&& x \geq 0 \\ &&& x_k \leq \lfloor x_k^0 \rfloor \\ P_2) \quad z^2 &= \min && c^T x \\ &s.a. && Ax \leq b \\ &&& x \geq 0 \\ &&& x_k \geq \lfloor x_k^0 \rfloor + 1 \end{aligned}$$

donde $\lfloor u \rfloor$ denota la parte entera de u .

Debido a que estamos restringiendo los problemas “hijos” con respecto al problema “padre”, se tendrá que $z^0 \leq z^1$ y $z^0 \leq z^2$. Adicionalmente, x^0 no es factible para ninguno de los problemas $P_1)$ o $P_2)$?

La siguiente figura ilustra el efecto de “cortar” usando el hecho que la coordenada k de la solución de la relajación lineal tiene un valor fraccionario: cualquier solución óptima entera al problema está contenida en alguna de las dos secciones que resultan después de la ramificación, pero no hay ninguna solución válida en la parte que fue eliminada.



La idea es, entonces, continuar resolviendo cada uno de los dos nuevos problemas por separado, ramificando nuevamente en caso que aún existan soluciones fraccionarias.

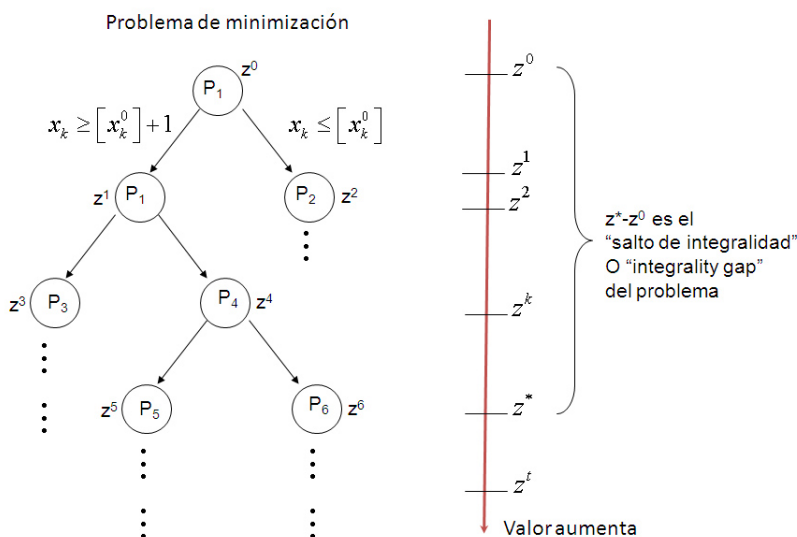
Se desarrolla entonces un “árbol binario” en que cada nodo es una división del anterior. Este proceso podría continuar, en el peor de todos los casos, hasta ramificar en cada una de los posibles valores de las variables. Esto puede llevar a un gran esfuerzo computacional, sin embargo es posible que durante el avance de la ramificación y exploración de los problemas resultantes puedan ocurrir algunos eventos con consecuencias que se describen a continuación:

- **Podemos encontrar un subproblema infactible.** En este caso, no tiene sentido continuar ramificando y toda la rama es “podada”.
- **Encontramos una solución entera** La pregunta que inmediata es si acaso esa es una solución óptima para IP. la respuesta es evidentemente negativa ya que podría haber otra solución entera en otra de las ramas del árbol y que no ha sido aún encontrada. Sin embargo, esta solución entera puede entregar información importante: Sea w el valor de la mejor solución entera encontrada hasta el momento. Llamaremos a este valor “incumbente”. Supongamos que en otro de los subproblemas generados por la ramificación encontramos una solución fraccionaria con valor z' y tal que $z' \geq w$. Dado que el valor óptimo de los subproblemas aumenta a medida que ramificamos, cualquier solución entera que se encuentre en la rama de ese problema tendrá un valor de función objetivo mayor o igual a z' y, por lo tanto, no es mejor que la solución entera actual cuyo valor es w . Por lo tanto, no es necesario ramificar ese subproblema y puede ser “podado”.

De este modo el árbol de ramificación se desarrolla con algunas ramas que se podan, ya sea por infactibilidad o por acotamiento. De ahí el nombre del procedimiento, que en Inglés se llama Branch and Bound.

La siguiente figura muestra el típico desarrollo del árbol de ramificación para un problema entero de minimización y como van cambiando los valores de las cotas que se van obteniendo. Inicialmente, el valor de la relajación lineal, z^0 es la cota inferior conocida al valor óptimo (desconocido) del problema, z^* . A medida que se va ramificando, se obtiene valores óptimos a los subproblemas en los nodos del árbol, $z^1, z^2, \dots$ los cuales cumplen con $z^0 \leq z^i$, para todo nodo i . Supongamos que en algún nodo, digamos t , se encuentra una solución entera con valor z^t . Como la solución es entera, es factible para IP) y podemos estar seguros que $z^* \leq z^t$. De este modo, aunque no conozcamos el valor óptimo del problema ya podemos dar un intervalo, $[z^0, z^t]$ en el cual este se encuentra, es decir:

$$z^0 \leq z^* \leq z^t.$$



Continuando la ramificación, puede ocurrir que encontremos otra solución entera con valor z^s . Si $z^s > z^t$ esta nueva solución no es mejor que la que ya tenemos. Sin embargo, si $z^s < z^t$, entonces sí tenemos una mejor solución entera y z^s pasa a ser el nuevo incumbente. En este caso, el intervalo en el cual estimamos se encuentra el valor óptimo es $[z^0, z^s]$, que es más pequeño que el anterior.

Pregunta: Supongamos que la búsqueda se detiene con la solución del nodo s . No tenemos una solución óptima pero sí una factible con valor z^s . ¿Cuánto es el error porcentual máximo estimado de esa solución con respecto al valor óptimo?

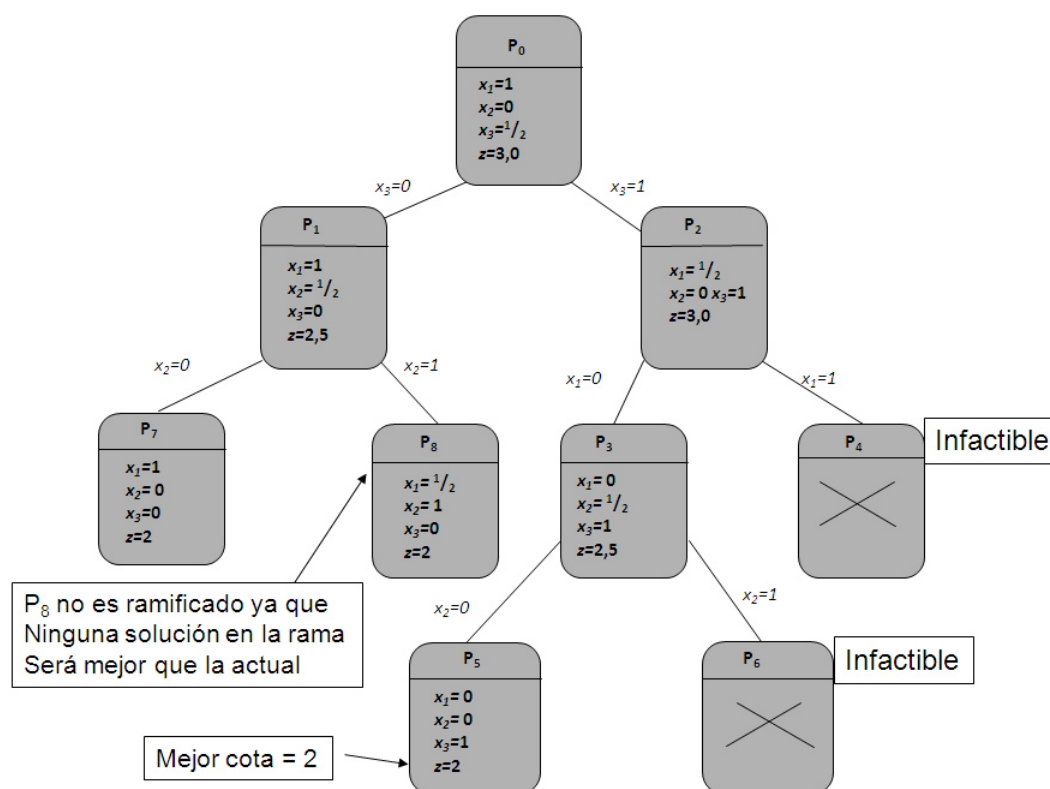
2.2 Ramificación y Acotamiento: ejemplo

Consideremos el siguiente problema:

$$\begin{aligned} \max \quad & 2x_1 + x_2 + 2x_3 \\ \text{s.t.} \quad & x_1 + 2x_2 + x_3 \leq 3 \\ & 2x_1 + 2x_2 + 2x_3 \leq 3 \\ & x_1 + 2x_2 + 2x_3 \leq 4 \\ & x_1, x_2, x_3 \in \{0, 1\} \end{aligned}$$

Este caso es especial en el sentido que es 0-1. las consecuencias de esto es que la ramificación se hace más fácil y tiene como consecuencia definir ramas en las cuales se fija la variable elegida en 0 ó en 1. (¿Por qué?). Adicionalmente, el problema de este ejemplo es de maximización lo que significa que los criterios indicados antes cambia en el sentido de las desigualdades.

Para comenzar, inicialmente no disponemos de ninguna solución entera que pueda servir de incumbente, por lo tanto este valor es igual a $-\infty$. La siguiente figura muestra el desarrollo del árbol en donde los nodos son generados en el orden indicado por sus números de identificación:



2.3 Ramificación y Acotamiento: algunas preguntas

Pueden surgir de inmediato una gran cantidad de preguntas sobre la operación del algoritmo de ramificación y acotamiento. A continuación presentamos algunas de ellas, con respuestas parciales. Más adelante en el curso volveremos repetidamente al proceso de Branch and Bound y ahondaremos en las respuestas a estas preguntas:

- **Cómo resolvemos los problemas que se generan de un nodo al siguiente?** Cada subproblema “hijo” se diferencia del “padre” en que tiene una restricción adicional (o una variable fijada). La solución del “padre” puede usarse como punto de partida para el SIMPLEX en el “hijo”. Si bien la solución del padre es infactible en el nodo hijo (ese es todo el sentido de agregar la restricción de ramificación), sí se puede crear fácilmente una solución que es factible para el problema dual del nodo.

De este modo es posible reoptimizar el problema pero usando el dual o, en forma más precisa, el algoritmo Simplex Dual.

- **¿En que orden generamos el árbol?:** En una etapa cualquiera habrá una lista de nodos pendientes de ser revisados. Una pregunta muy relevante es ¿qué problema seleccionamos para ramificar?. Existen dos estrategias de exploración totalmente contrapuestas:
 - “depth first” (en profundidad), en la cual se seleccionan sucesivamente nodos ramificando en forma persistente y haciendo crecer las ramas.
 - “breath first” (hacia lo ancho), en la cual se van resolviendo los nodos de modo de “llenar los niveles” del árbol.

Cada una de estas estrategias tienen argumentos a favor y en contra (¿cuáles se le ocurren a usted?). Los sistemas comerciales de software de optimización permiten combinarlas y el usuario puede especificar el grado de intensidad con el que se desea aplicar una o la otra.

- **Qué variables seleccionamos para ramificar?** Esta es otra gran pregunta. Debiera ser la que aporte más a “integralizar” la solución. Esto lleva a considerar distintos tipos de “prioridades” para las variables. Se puede priorizar por costo, por ejemplo, pero hay muchos otros criterios. Otra alternativa es usar procedimientos que tratan de “predecir” el grado de avance de la función objetivo cuando se ramifica en una u otra variable. Una de las técnicas sólidas para esto se conoce como “strong branching”, en la que se “prueba” el posible cambio de la función objetivo y se selecciona la variable más prometedora. Desde luego, hacer esto puede ser costoso computacionalmente debido a que hay que, igual, realizar alguna cantidad de iteraciones de Simplex hasta que haya una buena estimación del cambio de función objetivo. Esto se puede combinar también con la selección del problema a ramificar. Nuevamente el software comercial provee al usuario con parámetros que permiten controlar las diferentes estrategias.
- **Cuándo nos detenemos?** Idealmente en el óptimo, pero eso puede tardar mucho tiempo y tal vez debamos detenernos antes. En este caso es necesario usar las estimaciones al error en la solución actual evaluando el “salto de integralidad” entre la mejor solución entera obtenida hasta el momento y la mejor cota relajada disponible. Si ese error es aceptable dentro del sentido del problema que se está resolviendo, entonces la búsqueda puede detenerse con una solución factible con error máximo conocido.

Por otro lado, si el problema es infactible y su relajación lineal no lo es (lo que mostraría de inmediato la infactibilidad del problema entero), podría ser muy costoso llegar a la conclusión final de infactibilidad. No podemos detener el procedimiento hasta que se agoten todas las posibles ramificaciones y sean todas finalmente descartadas.

- Si tenemos una solución factible (no necesariamente óptima), podemos usarla como incumbente inicial? Efectivamente podemos hacer eso. Una solución factible al problema entero puede ser muy útil y una forma de encontrarla es mediante el uso de heurísticas. De hecho, el software comercial, como Gurobi y Cplex, tienen mecanismos heurísticos generales que tratan de encontrar soluciones factibles.

3 La importancia de una formulación “correcta” y la geometría de la Programación Entera

Recordemos el problema de localización de instalaciones y asignación de clientes que hemos mostrado ya en el curso:

$$\begin{aligned} \min \quad & \sum_{i=1}^m f_i y_i + \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.a.} \quad & \sum_{i=1}^m x_{ij} = 1, & j = 1, \dots, n \end{aligned} \quad (1)$$

$$\begin{aligned} & \sum_{j=1}^n x_{ij} \leq n y_i, & i = 1, \dots, m \\ & y_i, x_{ij} \in \{0, 1\} & i = 1, \dots, m; j = 1, \dots, n \end{aligned} \quad (2)$$

Una formulación alternativa, y totalmente equivalente para el problema, es:

$$\begin{aligned} \min \quad & \sum_{i=1}^m f_i y_i + \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.a.} \quad & \sum_{i=1}^m x_{ij} = 1, & j = 1, \dots, n \\ & x_{ij} \leq n y_i, & i = 1, \dots, m; j = 1, \dots, n \\ & y_i, x_{ij} \in \{0, 1\} & i = 1, \dots, m; j = 1, \dots, n \end{aligned} \quad (1)$$

Es fácil verificar con un ejemplo computacional que la segunda formulación puede resolverse en mucho menos tiempo que la primera, siendo que la segunda tiene muchas más restricciones. Esto parece contradictorio pero ilustra ciertas características de los problemas enteros que no son aparentes de inmediato. Existe una diferencia en la geometría subyacente de ambas formulaciones y eso hace que el Branch and Bound pueda tomar ciertas ventajas. A continuación exploraremos esa geometría.

3.1 Geometría de la Programación Entera

En el curso ya hemos revisado algunas nociones básicas de geometría poliedral. En particular, ya sabemos lo que son vértices y rayos y sabemos que un poliedro cualquiera puede ser representado de dos formas alternativas: mediante relaciones algebraicas, o como la suma de combinaciones convexas de los vértices del poliedro, más rayos en el cono de recesión (ver el material de geometría de Programación Lineal). Ahora daremos algunas definiciones adicionales, que son importantes en el contexto de Programación Entera.

Consideremos, entonces, un poliedro definido por un sistema

$$Ax \leq b$$

(las definiciones y resultados son válidos en otros formatos y se extienden de manera natural) El concepto que nos interesa es el de “cara” que corresponde a alguna de las estructuras externas. Formalmente:

Definición: Sea $P = \{x : Ax \leq b\}$ un poliedro y $F \subset P$. Se dice que F es una cara si y solo si

- $F \neq \emptyset$
- Existe un subsistema $A'x \leq b'$ de $Ax \leq b$ tal que

$$F = \{x \in P : A'x = b'\}$$

En palabras: una cara queda definida por algún subconjunto de las desigualdades, puestas en igualdad. Esto se cumple, efectivamente, para los puntos extremos, también para las “aristas” y para las caras planas que limitan el poliedro. Nos interesarán particularmente las caras “más grandes” del poliedro.

Definición: Sea P un poliedro y $F \subset P$ una cara. Se dice que F es una faceta de P si y solo si

- $F \neq P$
- No existe otra cara $F' \neq P$ tal que $F \subset F'$

Pregunta: ¿Cómo se relacionan facetas y desigualdades en este poliedro?

$$\begin{array}{rcl} x_1 & \leq & 1 \\ -x_1 & \leq & 0 \\ x_2 & \leq & 1 \\ -x_2 & \leq & 0 \end{array}$$

¿Y en este?

$$\begin{array}{rcl} x_1 & \leq & 1 \\ -x_1 & \leq & 0 \\ x_2 & \leq & 1 \\ -x_2 & \leq & 0 \\ x_1 + x_2 & \leq & 2 \end{array}$$

La pregunta es qué característica debe tener un conjunto mínimo de desigualdades en $Ax \leq b$ para que definan adecuadamente al poliedro.

Teorema: Si el poliedro $P = \{x : Ax \leq b\}$ es de dimensión completa y ninguna desigualdad es redundante, existe una correspondencia uno a uno entre las desigualdades y las facetas.

Determinar redundancia de una restricción puede verse simple, en términos geométricos o en dos variables, pero en muchas dimensiones no podemos “ver” nada. Sin embargo, es posible determinar redundancia mediante la factibilidad de un sistema alternativo que viene dado por el Lema de Farkas (ver el material de Programación Lineal).

3.2 Formulaciones de un problema entero

Consideremos, como antes, un problema entero de la forma

$$IP) \quad \begin{array}{ll} z^* = \min & c^T x \\ & s.t. \quad Ax \leq b \\ & \quad x \text{ entero} \end{array}$$

Sea $S = \{x \in \mathbb{R}^n : Ax \leq b, x \text{ entero}\}$ el conjunto de puntos enteros factibles para IP).

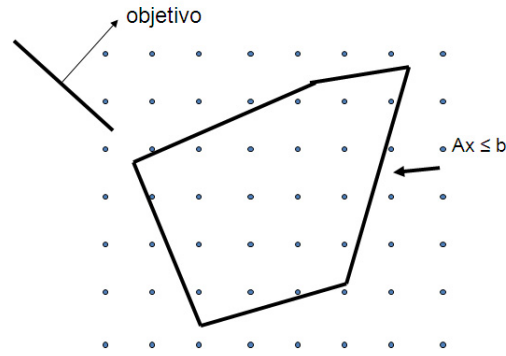
Definición: Un sistema lineal de desigualdades $Dx \leq d$ se dice que es una “formulación” para el problema entero IP) si

$$\{x \in \mathbb{R}^n : Dx \leq d, x \text{ entero}\} = S$$

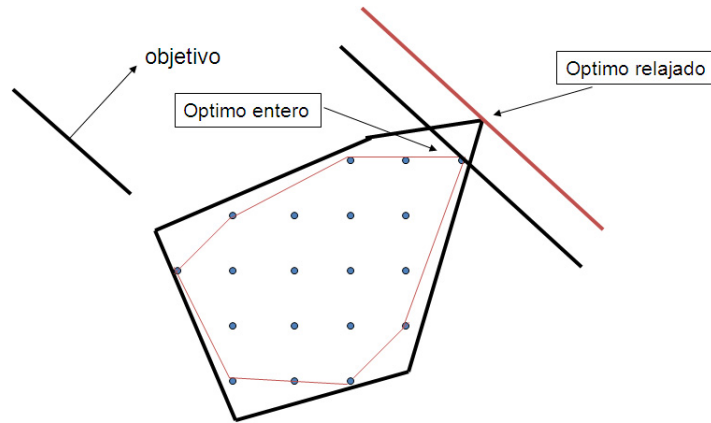
en palabras, es otra forma de escribir el mismo problema.

Vimos, con un ejemplo computacional, que usando distintas formulaciones para un mismo problema, obtenemos distintos “performances” en términos de facilidad de resolución, específicamente cuando un problema es escrito en dos formas equivalentes pero diferentes.

¿Cómo explicamos lo que está ocurriendo? La siguiente figura muestra el poliedro definido por las restricciones lineales de un problema, los puntos enteros y la dirección de la función objetivo.



De todos los puntos, sólo nos interesan los puntos enteros contenidos dentro del poliedro. Denotemos por $CH(S)$ la envoltura convexa del conjunto S , es decir de los puntos enteros contenidos en el poliedro. Por nuestros resultados de Teoría Poliedral, sabemos que esa envoltura convexa es un poliedro. Por lo tanto, existe una matriz D y vector d tales que $CH(S) = \{x : Dx \leq d\}$. Esto significa que podemos obtener la solución óptima de IP) resolviendo un problema de Programación Lineal sobre $CH(S)$.



Definición: Al sistema $Dx \leq d$ tal que $CH(S) = \{x : Dx \leq d\}$ se le llama “Formulación Ideal” del problema IP).

Como hemos visto, entonces, si tuviéramos a nuestra disposición la formulación ideal del problema podríamos, aparentemente, resolver nuestro problema de programación entera usando sólo Programación Lineal. Esto, en la realidad, no ocurrirá por las siguientes razones:

1. Puede ser imposible o muy difícil identificar la formulación ideal
2. Aún si fuera identificable (y se puede en algunos problemas), esta podría tener un número muy grande de restricciones y eso podría complicar su resolución mediante Programación Lineal.

Como recordaremos del punto anterior, las facetas de un poliedro son las caras “límite” de este. De este modo, la formulación ideal está generalmente descrita por desigualdades (restricciones) que definen facetas. Si bien puede resultar imposible encontrar todas esas restricciones, podemos observar que realmente nos interesan sólo aquellas que se encuentran en la vecindad de la solución óptima. De esta manera, aún puede ser de utilidad conocer algunas de las desigualdades que definen facetas de la envoltura convexa de los puntos enteros factibles del problema. Una parte importante de los desarrollos en Programación Entera en las últimas décadas ha sido precisamente la identificación de este tipo de restricciones para algunos problemas específicos.

Si bien la formulación ideal es la más importante, un problema entero puede admitir muchas formulaciones, todas válidas.

Definición: Dadas dos formulaciones válidas para el problema IP), la F1 dada por el sistema $D^1x \leq d^1$ y la F2, dada por el sistema $D^2x \leq d^2$, decimos que F1 “más fuerte” que F2 si

$$\{x \in R^n : D^1x \leq d^1\} \subset \{x \in R^n : D^2x \leq d^2\}$$

La más fuerte de todas las formulaciones es la Formulación Ideal, pero, como decíamos antes, encontrar la formulación ideal es muy difícil o casi imposible.

Sin embargo, lo que sí tiene sentido es tratar de aproximarse a la formulación ideal. Es decir, usar una formulación “fuerte” de modo que aproxime mejor la envoltura convexa $CH(S)$. Los “triggers” que usamos en los ejemplos y el ajuste del parámetro M ayudan precisamente a fortalecer la formulación inicial del problema. Veremos más adelante que pueden diseñarse mecanismos sistemáticos para fortalecer la formulación de un problema entero. Más aún, estos mecanismos pueden combinarse con el algoritmo de Ramificación y Acotamiento.

Discutamos ahora cómo medir la “fortaleza” de una formulación comparada con otra. Sean F1 y F2 dos formulaciones, con sistemas $D^1x \leq d^1$ y $D^2x \leq d^2$.

Sean

$$z^1 = \max\{c^T x : D^1x \leq d^1\}$$

y

$$z^2 = \max\{c^T x : D^2x \leq d^2\}$$

Entonces, si F1 es más fuerte que F2, se tiene que

$$z^1 \leq z^2.$$

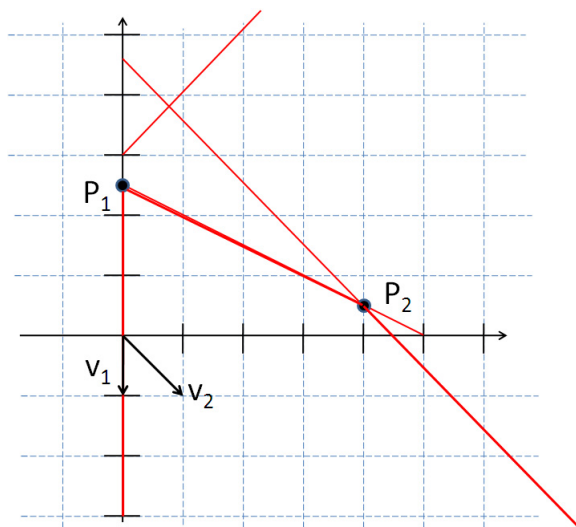
Entonces, la mejora en el valor de la relajación lineal es un indicador de cómo se fortalece la formulación, tal como ocurría en el ejemplo del Branch and Bound.

Ejemplo:

Considere el poliedro P definido por el siguiente sistema de desigualdades lineales:

$$\begin{array}{rcl} 2x_1 + 2x_2 & \leq & 9 \\ x_1 + 2x_2 & \leq & 5 \\ x_1 & \geq & 0 \\ -x_1 + x_2 & \leq & 3 \end{array}$$

La siguiente figura muestra las restricciones y la región factible resultante:



Este poliedro no es acotado, pero en este ejemplo ilustraremos que los conceptos de formulaciones se extienden igual. Usamos el supuesto de que los poliedros eran acotados únicamente para simplificar.

Hay dos puntos extremos, P_1 y P_2 cuya envoltura convexa define el polítopo C y dos direcciones extremas, v_1 y v_2 cuya envoltura cónica define el cono de recesión K . Estos son:

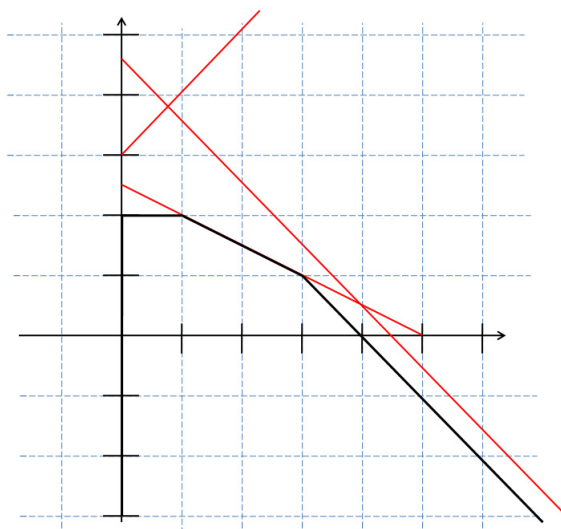
$$P_1 = \begin{pmatrix} 0 \\ 2,5 \end{pmatrix}, P_2 = \begin{pmatrix} 4 \\ 0,5 \end{pmatrix}, v_1 = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, v_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Se puede ver que en este caso, no existe correspondencia entre las restricciones algebraicas y las facetas, y la razón es que la cuarta restricción es redundante como se ve claramente del dibujo.

Considere ahora el siguiente problema de programación lineal entera:

$$\begin{array}{ll} \max & x_2 \\ \text{s.a.} & (x_1, x_2) \in P \\ & x_1, x_2 \text{ enteros.} \end{array}$$

La siguiente figura muestra la formulación ideal:



El sistema de desigualdades que la define es:

$$\begin{array}{rcl} x_1 + x_2 & \leq & 4 \\ x_1 + 2x_2 & \leq & 5 \\ x_1 & \geq & 0 \\ x_2 & \leq & 2 \end{array}$$

y para este sistema sí existe correspondencia entre desigualdades y facetas del poliedro resultante de la envoltura convexa y su extensión con el cono.

3.3 Unimodularidad

Consideremos el siguiente problema, conocido como problema de asignación (y que ustedes deben conocer).

$$\begin{array}{ll} \min & \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.a.} & \sum_{j=1}^n x_{ij} = 1, i = 1 \dots, n \\ & \sum_{i=1}^n x_{ij} = 1, j = 1 \dots, n \\ & x_{ij} \in \{0, 1\}, i, j = 1, \dots, n \end{array}$$

Si resolvemos cualquier instancia de este problema podremos ver que este problema tiene la propiedad de que la relajación lineal entrega ya valores enteros. ¿Por qué ocurre esto?

Definición: Sea A una matriz de $m \times n$. Se dice que A es unimodular si el valor de cualquier subdeterminante de A es 1, 0 ó -1.

Teorema: Consideremos el problema lineal $\min\{c^T x : Ax = b, x \geq 0\}$. Supongamos que A es unimodular. Entonces, si b tiene componentes enteras, toda solución básica del problema tiene coordenadas enteras.

Demostración:

Esto se sigue del hecho que un punto extremo es solución de un sistema de ecuaciones correspondiente a una solución básica factible. Más específicamente, sea x_B la parte básica asociada a un punto extremo. Entonces, existe una submatriz cuadrada de A , B , tal que B es invertible y $Bx_B = b$. Ahora recordemos que un sistema lineal se puede resolver desarrollando por determinantes. la componente i de x_b satisface:

$$x_{Bi} = \frac{\det(\bar{B}_i)}{\det(B)}$$

donde $\bar{B}_i$ denota la matriz resultante de reemplazar la i -ésima columna por el vector b . Tenemos que por hipótesis, $\det(B)$ es igual a 1 ó -1. Por otro lado, si b tiene componentes enteras, el desarrollo por subdeterminantes muestra de inmediato que $\det(\bar{B}_i)$ es un número entero. Luego, x_{Bi} es un número entero.

En el problema de asignación, la matriz es unimodular. Más aún, toda matriz que provenga de una estructura de problema de flujo en redes es unimodular (lo veremos pronto).

La unimodularidad es una propiedad muy importante, si se tiene. Se conocen estructuras que poseen la propiedad. Por ejemplo, todo problema cuyas restricciones correspondan a flujo en redes, poseen esta propiedad (el problema de asignación es, en realidad, un problema de flujo en redes). Desafortunadamente, la mayoría de los problemas enteros no tienen esta propiedad y, por lo tanto, resultarán muy difíciles de resolver.

4 Planos Cortantes para resolver problemas enteros

Como ya hemos visto, el tener una formulación que se aproxime bien a la formulación ideal del problema es muy importante para aumentar las probabilidades de resolver el problema de manera eficiente. En este capítulo veremos en más detalle cómo hacer esto en forma sistemática. Las técnicas de Planos Cortantes son uno de los enfoques que han tenido gran éxito en la resolución de problemas difíciles del ámbito de la programación entera.

4.1 Preliminares

Consideremos un problema entero de la forma:

$$(IP) \quad \begin{array}{ll} z^* = \min & c^T x \\ & s.a. \quad Ax \leq b \\ & \quad x \text{ entero} \end{array}$$

Sea $S = \{x : Ax \leq b, \text{ entero}\}$ la región factible del problema. Ya hemos visto lo importante de fortalecer las formulaciones. Ahora veremos como hacerlo en forma más sistemática.

Definición: $\alpha^T x \leq \beta$ se dice que es una desigualdad válida para S si

$$x \in S \implies \alpha^T x \leq \beta$$

Supongamos resolvemos la relajación lineal del problema:

$$P_0) \quad z^0 = \min_{s.a.} \quad c^T x \quad Ax \leq b.$$

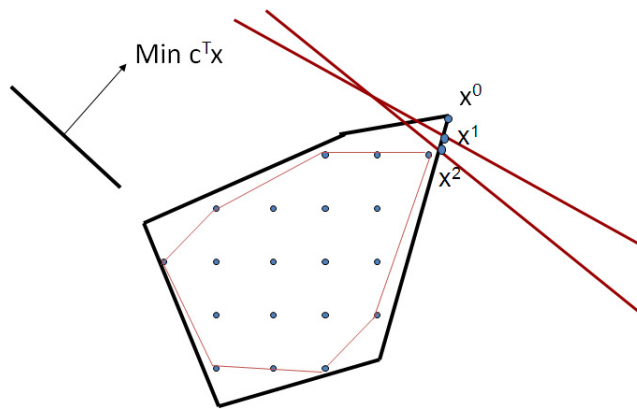
Sea x^0 solución óptima de la relajación lineal. La idea es construir una desigualdad válida $(\alpha^1)^T x \leq \beta^1$ tal que

$$(\alpha^1)^T x^0 > \beta^1$$

Esta nueva restricción “corta” la solución relajada pero, por ser válida, no excuye ninguna solución factible del problema entero original. Ahora resolvemos el problema:

$$P_1) \quad z^1 = \min_{s.a.} \quad c^T x \quad Ax \leq b \quad (\alpha^1)^T x \leq \beta^1.$$

y tenemos una nueva solución x^1 y valor z^1 tales que $z^* \geq z^1 \geq z^0$ (idealmente, $z^1 > z^0$). La idea es hacer esto sistemáticamente. La figura ilustra la estrategia.



Un algoritmo general de planos cortantes:

0. Sea $S_0 = \{x : Ax \leq b\}$, $k = 0$.

1. Etapa general k :

- Resolver el problema lineal $\min\{c^T x : x \in S_k\}$.
- Sea x_k solución óptima.
- Si x_k es entero, PARAR, tenemos una solución óptima (o se satisface otro criterio de parada).

2. Si no, buscar una desigualdad válida $\alpha_k^T x \leq \beta_k$ tal que $\alpha_k^T x_k > \beta_k$.

3. $S_{k+1} = S_k \cap \{x : \alpha_k^T x \leq \beta_k\}$.

4. $k = k + 1$, ir a 1.

Una pregunta inmediata que surge de lo anterior es cómo generar planos cortantes. Un procedimiento que genera planos cortantes es lo que ya hemos llamado un “oráculo de separación”. Hay dos enfoques para generar planos cortantes:

Procedimientos generales: El primero y más famoso es el algoritmo de planos cortantes desarrollado por R. Gomory. Puede ser usado en cualquier problema entero y garantiza encontrar una solución óptima después de un número finito (eventualmente muy grande) de cortes. Estudiaremos este algoritmo un poco más adelante. Otros procedimientos también se pueden sistematizar para generar otros tipos de cortes. Estos son los llamados Covers, Flow Covers y cortes basados en “convexificación”. También estudiaremos

más adelante algunos de estos.

Cortes especializados que usan la estructura del problema. Estos cortes se construyen en forma especializada según el problema que se está resolviendo. Veremos más adelante un ejemplo de esto aplicado al Problema del Vendedor Viajero.

4.2 Métodos de Planos Cortantes Generales

Como ya hemos comentado, sería de gran utilidad poder generar desigualdades válidas en forma sistemática para cualquier problema, ojalá en forma “automática”. Veremos ahora que eso se puede hacer bajo ciertas circunstancias.

Comenzaremos por ver que las propiedades de los números permiten generar desigualdades válidas. Consideremos, por ejemplo, la siguiente desigualdad:

$$3x_1 + 5x_2 \leq 10, x_i \in \{0, 1\}, i = 1, 2$$

Esta restricción puede ser obviamente fortalecida reemplazando el lado derecho por 8, ya que ese es el menor valor que puede llegar a tomar el lado izquierdo de la desigualdad en cualquier circunstancia.

Consideremos ahora el conjunto:

$$P = \{x \in \mathbb{Z}_+^4 : 13x_1 + 20x_2 + 11x_3 + 6x_4 \geq 72\}$$

Dividamos la desigualdad por, digamos, 11:

$$\frac{13}{11}x_1 + \frac{20}{11}x_2 + x_3 + \frac{6}{11}x_4 \geq 6\frac{6}{11}$$

Ahora, si redondeamos cada coeficiente del lado izquierdo al entero superior la desigualdad sigue siendo válida:

$$2x_1 + 2x_2 + x_3 + x_4 \geq 6\frac{6}{11}$$

Y como los x_i son enteros, podemos “apretar” el lado derecho a 7:

$$2x_1 + 2x_2 + x_3 + x_4 \geq 7$$

Tomemos ahora los siguientes valores fraccionarios: $\bar{x}_1 = 0$, $\bar{x}_2 = 1.8$, $\bar{x}_3 = 3.28$, $\bar{x}_4 = 0$. Tenemos que:

$$13\bar{x}_1 + 20\bar{x}_2 + 11\bar{x}_3 + 6\bar{x}_4 = 72.08 \geq 72.$$

Luego, $\bar{x}$ es factible para la relajación lineal de la definición de P . Pero:

$$2\bar{x}_1 + 2\bar{x}_2 + \bar{x}_3 + \bar{x}_4 = 6.88 \not\geq 7$$

Luego, la restricción que hemos construido “corta” un punto de la relajación lineal de la definición de P , sin excluir ningún punto de P .

Notemos que obtuvimos la desigualdad anterior por “simple” manipulación aritmética. Esto nos lleva a preguntarnos por qué haber dividido por 11 y si acaso existe alguna forma más sistemática de hacer lo anterior. La respuesta es sí y una forma de hacerlo es el procedimiento que fue desarrollado en 1958 por Ralph Gomory (en IBM) y conocido como Algoritmo de Gomory.

4.3 Soluciones en enteros

Antes de pasar a los detalles del algoritmo de Gomory es interesante revisar algunos conceptos que están en su base. Sabemos que el problema: dados (A, b) en números racionales, determinar si

$$Ax \leq b, x \in \mathbb{Z}^n$$

es factible es **NP-Completo**. Por otro lado, el problema de determinar si

$$Ax = b, x \in \mathbb{Z}^n$$

es factible es resoluble en tiempo polinomial! En verdad es sorprendente que la existencia o no de desigualdades hace la diferencia en complejidad. (Nota: no basta agregar variables de holgura a las desigualdades, ya que igual nos quedamos con desigualdades en variables).

La verdad es que estos son resultados más antiguos que la Programación Entera. Encontrar una solución en enteros para $Ax = b$ (con A y b enteros) es un caso particular de Ecuación Lineal de Diofanto y procedimientos para encontrar soluciones a estas ecuaciones fueron desarrollados por Brahmagupta, un matemático indio, alrededor del 600 D.C. Estos procedimientos se relaciona con el Algoritmo Euclideano y la descomposición en fracciones continuas.

Nota: el problema de encontrar soluciones en enteros a una ecuación general (polinomios de grado > 1) de Diofanto es el décimo problema de Hilbert y se probó “indecidible” (en el sentido del Teorema de Goedel) en 1970 por Yuri Matiyasevich (y otros).

Existe también un análogo al Lema de Farkas, pero en el caso entero:

Lema de Farkas Entero: Sean $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^m$. El sistema $Ax = b, x \in \mathbb{Z}^n$ tiene solución si y sólo si el sistema $A^T y = 0, b^T y < 0, y \in \mathbb{Q}^m$ no tiene solución.

A partir de esto puede incluso construirse una teoría de dualidad para Programación Entera (no la estudiaremos, y no es igual a la dualidad lagrangiana).

Tenemos entonces que podemos resolver la relajación lineal del problema entero con el SIMPLEX. Por otro lado, sabemos que El SIMPLEX resuelve sistema de ecuaciones. Tal vez sea posible inducir soluciones enteras a esos sistemas de ecuaciones. Eso es lo que está en la base de la conceptualización de Gomory.

4.4 Cortes de Gomory

Supongamos que el problema original está dado en forma estándar:

$$\begin{array}{ll} \max & c^T x \\ \text{s.a.} & Ax = b \\ & x \geq 0, \text{ entero} \end{array}$$

Supongamos resolvemos la relajación lineal y obtenemos una solución óptima x^0 . Supongamos también que x^0 es fraccionario. Por otro lado, x^0 es solución básica factible. Entonces, podemos asumir que A se particiona en la forma

$$A = [B|N]$$

donde B es la “base”, matriz invertible. Del mismo modo, x se particiona como

$$x = (x_B, x_N),$$

donde $x_B = B^{-1}b \geq 0, x_N = 0$.

Sea NB el conjunto de índices de las variables no básicas. Por la escritura básica del sistema de restricciones se tiene:

$$Ax = b \Leftrightarrow Bx_B + Nx_N = b \Leftrightarrow x_B + B^{-1}Nx_N = B^{-1}b$$

En particular, para x^0 :

$$(x_B^0)_i + \sum_{j \in NB} \bar{a}_{ij}(x_N^0)_j = \bar{b}_i, \quad i = 1, \dots, m.$$

(donde $\bar{a}_{ij}$ son coeficientes de $B^{-1}N$ y $\bar{b} = B^{-1}b$). Si la solución no es entera, entonces existe un índice k tal que $\bar{b}_k$ es fraccionario.

La ecuación k general es:

$$(x_B)_k + \sum_{j \in NB} \bar{a}_{kj}(x_N)_j = \bar{b}_k. \quad (*)$$

y debe ser cumplida por los puntos enteros factibles. Además, como $x \geq 0$ y $\bar{b}_k$ es fraccionario tenemos que

$$(x_B)_k + \sum_{j \in NB} \lfloor \bar{a}_{kj} \rfloor (x_N)_j \leq \lfloor \bar{b}_k \rfloor$$

es una desigualdad válida. Restando de (*) obtenemos:

$$\sum_{j \in NB} (\bar{a}_{kj} - \lfloor \bar{a}_{kj} \rfloor)(x_N)_j \geq (\bar{b}_k - \lfloor \bar{b}_k \rfloor).$$

Que es también una desigualdad válida para la formulación entera. Sin embargo, podemos ver de inmediato que x^0 no satisface esta desigualdad, ya que los valores de las variables básicas son nulos y el lado derecho resultó $\neq 0$. Este es el corte de Gomory extraído de una solución fraccionaria de Programación lineal. Este corte se agrega al problema relajado original y se resuelve nuevamente. Si hay nuevamente solución fraccionaria, se genera un nuevo corte a partir de la solución de Programación Lineal, y así sucesivamente.

Lo anterior es básicamente el algoritmo que desarrolló Gomory en 1958. Se puede mostrar convergencia en un número finito de etapas (aunque exponencialmente grande) Sin embargo, en sus orígenes se le considero muy ineficiente y lento, pero hoy es posible manejar cortes de Gomory y sus derivados de manera computacional y numérica muy eficiente. De hecho, Gurobi y otros sistemas los generan en forma automática.

Ejemplo: (tomado de L. Wolsey, *Integer Programming*, pag. 125)

$$\begin{aligned} z = \max \quad & 4x_1 - x_2 \\ & 7x_1 - 2x_2 \leq 14 \\ & x_2 \leq 3 \\ & 2x_1 - 2x_2 \leq 3 \\ & x_1, x_2 \geq 0 \text{ and integer.} \end{aligned}$$

Agregamos variables de holgura que también deben ser enteras. Al optimizar linealmente queda:

$$\begin{aligned} z = \max \quad & \frac{59}{7} \\ & x_1 \quad -\frac{4}{7}x_3 - \frac{1}{7}x_4 = \frac{20}{7} \\ & x_2 \quad +\frac{1}{7}x_3 + \frac{2}{7}x_4 = 3 \\ & \quad -\frac{2}{7}x_3 + \frac{10}{7}x_4 + x_5 = \frac{23}{7} \\ & x_1, x_2, x_3, x_4, x_5 \geq 0 \text{ and integer.} \end{aligned}$$

La solución relajada es $x = (20/7, 3, 0, 0, 23/7)$. Usamos la primera fila donde x_1 es fraccionaria y se genera el corte:

$$\frac{1}{7}x_3 + \frac{2}{7}x_4 \geq \frac{6}{7}$$

o:

$$\frac{1}{7}x_3 + \frac{2}{7}x_4 - s = \frac{6}{7}$$

(donde s es una nueva variable de holgura). Al reoptimizar queda el siguiente sistema:

$$\begin{aligned} z = \max \quad & \frac{15}{2} \\ & x_1 \quad -\frac{1}{2}x_5 - 3s = 2 \\ & x_2 \quad -\frac{1}{2}x_5 + s = \frac{1}{2} \\ & x_3 \quad -x_5 - 5s = 1 \\ & x_4 \quad +\frac{1}{2}x_5 + 6s = \frac{5}{2} \\ & x_1, x_2, x_3, x_4, x_5, s \geq 0 \text{ and integer.} \end{aligned}$$

La solución relajada es $x = (2, 1/2, 1, 5/2, 0)$. Usamos la segunda fila donde x_2 es fraccionaria y se genera el corte:

$$\frac{1}{2}x_5 \geq \frac{1}{2}$$

o:

$$\frac{1}{2}x_5 - t = \frac{1}{2}$$

(donde t es la nueva variable de holgura). Al reoptimizar queda el siguiente sistema:

$$\begin{array}{rcccccccl} z = \max & 7 & & & & & -3s & -t & & \\ & x_1 & & & & & +s & & & = 2 \\ & & x_2 & & & & +s & -t & & = 1 \\ & & & x_3 & & & -5s & -2t & & = 2 \\ & & & & x_4 & & +6s & +t & & = 2 \\ & & & & & x_5 & & -t & & = 1 \\ & x_1, & x_2, & x_3, & x_4, & x_5, & s, & t & \geq 0 \text{ and integer.} \end{array}$$

que tiene solución entera.

4.5 Cortes de Chvatal-Gomory

Lo que hemos mostrado hasta ahora son casos particulares de un procedimiento más general. Consideremos el conjunto $S = \{x : Ax \leq b, x \geq 0, x \text{ entero}\}$. Sea $a_1, \dots, a_n$ las columnas de A .

Procedimiento de Chvatal-Gomory (C-G):

- Sea $u = (u_1, \dots, u_m) \geq 0$.
- La desigualdad

$$\sum_{j=1}^n \lfloor u^T a_j \rfloor x_j \leq \lfloor u^T b \rfloor$$

es válida para S .

Teorema: Toda desigualdad válida para S puede obtenerse aplicando C-G un número finito de veces.

Corolario: La formulación ideal para S puede obtenerse aplicando el procedimiento de C-G un número finito de veces.

Notemos la analogía que existe con la generación de desigualdades redundantes para un sistema lineal (continuo): basta hacer combinaciones lineales de las ecuaciones existentes. En este caso, esas combinaciones deben ser además “redondeadas”. Por supuesto, el resultado no nos dice nada respecto a qué tan fácil o difícil es encontrar la combinación específica que da origen a las mejores desigualdades válidas. De hecho, suele ser muy difícil.

Pregunta: ¿Qué estaríamos demostrando si logramos una forma eficiente (en tiempo polinomial) de aplicar el procedimiento C-G para encontrar facetas de un problema entero? ¿Por qué?

4.6 Desigualdades de “Covers”

Consideremos un conjunto “tipo knapsack”:

$$S = \{x : \sum_{j=1}^n a_j x_j \leq b, x_j \in \{0, 1\}, j = 1, \dots, n\}$$

Observamos que siempre podemos asumir $a_j > 0, j = 1, \dots, n$ y $b > 0$.

Definición: $C \subset N$ se llama “cover” (o cubrimiento) si

$$\sum_{j \in C} a_j > b$$

Proposición: Si C es un cover, entonces la desigualdad

$$\sum_{j \in C} x_j \leq \text{card}(C) - 1$$

es válida para S .

Ejemplo: busquemos covers en la siguiente desigualdad:

$$11x_1 + 6x_2 + 6x_3 + 5x_4 + 5x_5 + 4x_6 + x_7 \leq 19, x_j \in \{0, 1\}, j = 1, \dots, 7.$$

En el ejemplo, $C = \{3, 4, 5, 6\}$ es un cover y la desigualdad $x_3 + x_4 + x_5 + x_6 \leq 3$ es válida para

$$S = \{x : 11x_1 + 6x_2 + 6x_3 + 5x_4 + 5x_5 + 4x_6 + x_7 \leq 19, x_j \in \{0, 1\}, j = 1, \dots, 7\}$$

Notemos, sin embargo, que también es válida para

$$S' = \{x : 6x_3 + 5x_4 + 5x_5 + 4x_6 \leq 19, x_j \in \{0, 1\}, j = 1, \dots, 7\}$$

Notemos también que $S \subset S'$, de hecho, S' es mucho más grande. Esto significa que la desigualdad $x_3 + x_4 + x_5 + x_6 \leq 3$ posiblemente no es un muy “ajustada” para S . Veamos cómo solucionar este problema. Necesitamos tratar de incorporar las otras variables en la restricción. Consideremos la restricción:

$$\alpha_1 x_1 + x_3 + x_4 + x_5 + x_6 \leq 3$$

Pregunta: ¿existirá algún valor de α_1 tales que esa desigualdad es válida para el conjunto:

$$S'' = \{x : 11x_1 + 6x_3 + 5x_4 + 5x_5 + 4x_6 \leq 19, x_j \in \{0, 1\}, j = 1, \dots, 7\} ?$$

(notar que este incluye la variable x_1). Haremos el análisis, el cual depende solamente de los dos posible valores de x_1 : $x_1 = 1$ y $x_1 = 0$.

Si $x_1 = 0$, la desigualdad es válida para cualquier valor de α_1 .

Si $x_1 = 1$: Hay que determinar α_1 de modo que

$$\alpha_1 + x_3 + x_4 + x_5 + x_6 \leq 3$$

sea válida para cualquier punto 0-1 que cumple

$$11 + 6x_3 + 5x_4 + 5x_5 + 4x_6 \leq 19$$

o, equivalentemente:

$$6x_3 + 5x_4 + 5x_5 + 4x_6 \leq 8$$

Planteamos el siguiente problema:

$$\begin{aligned} w = \max \quad & x_3 + x_4 + x_5 + x_6 \\ \text{s.a.} \quad & 6x_3 + 5x_4 + 5x_5 + 4x_6 \leq 8 \\ & x_j \in \{0, 1\}, j = 3, 4, 5, 6. \end{aligned}$$

El valor óptimo es $w = 1$. Podemos, entonces, elegir $\alpha_1 = 3 - w = 2$. La desigualdad válida es ahora:

$$2x_1 + x_3 + x_4 + x_5 + x_6 \leq 3$$

Podemos repetir ahora el análisis para las otras variables.

Este procedimiento se llama “levantamiento” o “lifting”.

Proposición: Para una restricción tipo “knapsack”, si partimos con un cover C cuya cardinalidad sea mínima, $a_j < b, j = 1, \dots, n$ y aplicamos el procedimiento de “lifting” repetidamente, se obtendrá una desigualdad válida que define una faceta para el problema de mochila.

El problema de separación para “Covers”

Dada una solución fraccionaria, ¿cómo podemos identificar un cover violado? Consideremos la restricción

$$\sum_{j=1}^n a_j x_j \leq b, x_j \in \{0, 1\}, j = 1, \dots, n$$

Supongamos que tenemos una solución fraccionaria $\bar{x}$. Primero observemos que la desigualdad de cover

$$\sum_{j \in C} x_j \leq \text{card}(C) - 1$$

se puede escribir

$$\sum_{j \in C} x_j < \text{card}(C) \Leftrightarrow \sum_{j \in C} x_j < \sum_{j \in C} 1 \Leftrightarrow \sum_{j \in C} (1 - x_j) \geq 1$$

Entonces, tenemos que encontrar $C \subset N$ tal que

$$\sum_{j \in C} (1 - \bar{x}_j) < 1$$

Para esto plantearemos un problema de optimización entera. Sea $z_j \in \{0, 1\}$ tal que $z_j = 1$ si el índice j está en el cover, 0 si no. Entonces, formulamos el siguiente problema:

$$\begin{aligned} \bar{z} = \min \quad & \sum_{j=1}^n z_j (1 - \bar{x}_j) \\ \text{s.a.} \quad & \sum_{j=1}^n z_j a_j > b \\ & z_j \in \{0, 1\}, j = 1, \dots, n. \end{aligned}$$

(Si b es entero la restricción se puede plantear como $\geq b - 1$.) Si $\bar{z} < 1$, hemos encontrado un cover que “corta” la solución actual.

De este modo, la separación de Covers requiere la solución de un problema de mochila. Y lo mismo ocurre con el procedimiento de “lifting”. Estrictamente, el problema de mochila es NP-Completo, pero se puede resolver igual de manera eficiente. Es importante aquí recordar de todos modos que no necesitamos la optimalidad: basta cumplir la condición de corte.

Ejemplo:

Consideremos la siguiente desigualdad del tipo “mochila”:

$$3x_1 + x_2 + 5x_3 + 2x_4 \leq 6, x_j \in \{0, 1\}, j = 1, \dots, 4.$$

Supongamos que se dispone de la siguiente solución fraccionaria asociada a la restricción anterior: $x_1 = 2/3$, $x_2 = 1/2$, $x_3 = 1/2$, $x_4 = 1/2$. Formule el problema de optimización correspondiente al problema de separación para restricciones de “covers”, y usando ese problema encuentre una restricción de cover que “corte” la solución fraccionaria. Debe formular el problema de separación, no basta con que indique un cover “a ojo”.

Ejemplo:

Consideremos ahora la siguiente desigualdad de tipo “mochila”:

$$11x_1 + 6x_2 + 6x_3 + 5x_4 + 5x_5 + 4x_6 + x_7 \leq 19, x_j \in \{0, 1\}, j = 1, \dots, 7.$$

Los índices $\{1, 3, 5, 6\}$ identifican un “cover” para esa desigualdad. Determine si acaso es posible realizar “lifting” a esa desigualdad de cover, usando la variable x_2 , indicando con claridad por qué sí o por qué no.

4.7 Desigualdades válidas específicas a un problema: “triggers”

Los cortes que hemos mostrado en la subsección anterior son generales, en el sentido que están definidos para cualquier problema. Muchas veces, sin embargo, es posible utilizar la estructura específica del problema y con eso construir desigualdades válidas específicas al problema. Vamos a ilustrar lo anterior con el siguiente problema de Loteo Dinámico Capacitado, o de planificación de producción con costos fijos.

Una empresa produce n productos y enfrenta un horizonte de T periodos. Existen demandas d_{it} , para el producto i en el periodo T . En cada periodo de tiempo se puede decidir producir o no. Lo que no se consume queda en inventario para periodos futuros. Si en un periodo se produce, se incurre (además del costo variable de producción) en un costo fijo de “set-up”, independientemente del número de productos diferentes producidos y de las cantidades. Sea c_{it} el costo de producir una unidad de i en T ; F_t el costo fijo de producir algún producto en t . Además, sea K_t la capacidad productiva disponible en el periodo t , medida en horas; cada unidad del producto i consume a_i horas de capacidad. Sea I_{i0} el inventario inicial en mano para el producto i y sea h_{it} el costo de inventario para el producto i en el periodo t . Sea x_{it} , la producción del producto i en t ; I_{it} , el inventario que queda del producto i al final del periodo t ; y variables binarias z_t que valen 1 si se produce algún producto en el periodo t , y 0 en caso contrario. El modelo que minimiza costos, satisface demandas y no excede capacidad es:

$$\begin{aligned} \min \quad & \sum_{t=1}^T \left\{ F_t z_t + \sum_{i=1}^n c_{it} x_{it} \right\} \\ \text{s.a.} \quad & I_{it} = I_{i,t-1} + x_{it} - d_{it} \quad i = 1, \dots, n; t = 1, \dots, T \quad (1) \\ & \sum_{i=1}^n a_i x_{it} \leq K_t \quad t = 1, \dots, T \quad (2) \\ & x_{it} \leq M z_t \quad i = 1, \dots, n; t = 1, \dots, T \quad (3) \\ & x_{it} \geq 0, I_{it} \geq 0, z_t \in \{0, 1\} \quad i = 1, \dots, n; t = 1, \dots, T \quad (4) \end{aligned}$$

Observemos que de las restricciones (2) se puede deducir que siempre se cumple:

$$x_{it} \leq \frac{K_t}{a_i}, \quad i = 1, \dots, n; t = 1, \dots, T$$

Por lo tanto, una de las primeras cosas que se puede hacer es reemplazar el parámetro “M grande” por $\frac{K_t}{a_i}$ y reescribir el modelo como:

$$\begin{aligned} \min \quad & \sum_{t=1}^T \left\{ F_t z_t + \sum_{i=1}^n c_{it} x_{it} \right\} \\ \text{s.a.} \quad & I_{it} = I_{i,t-1} + x_{it} - d_{it} \quad i = 1, \dots, n; t = 1, \dots, T \quad (1) \\ & \sum_{i=1}^n a_i x_{it} \leq K_t \quad t = 1, \dots, T \quad (2) \\ & x_{it} \leq \frac{K_t}{a_i} z_t \quad i = 1, \dots, n; t = 1, \dots, T \quad (3) \\ & x_{it} \geq 0, I_{it} \geq 0, z_t \in \{0, 1\} \quad i = 1, \dots, n; t = 1, \dots, T \quad (4) \end{aligned}$$

Esto es importante ya que, tener el M lo más ajustado posible ayuda a que la formulación sea más fuerte.

Ahora bien, tenemos que para un periodo t específico, la producción para el futuro debe ser al menos la necesaria para cubrir la demanda que aún no ha sido satisfecha. Es decir,

$$\sum_{j=t}^T x_{ij} \geq \sum_{j=t}^T d_{ij}$$

Y usando ahora las restrcciones (3), obtenemos restricciones (redundantes) de la forma:

$$\sum_{j=t}^T z_j \left(\frac{K_t}{a_i} \right) \geq \sum_{j=t}^T d_{ij}, \quad i = 1, \dots, n; t = 1, \dots, T$$

Esta restricción es totalmente redundante a la formulación de problema pero puede tener un impacto importante ya que podría estar “cortando” soluciones fraccionarias. Esto es especialmente cierto si en la relajación lineal las variables z_j , en su versión continua, toman valores fraccionarios muy cercanos a 0. De ese modo, el lado izquierdo de la restricción puede ser muy pequeño comparado con la demanda que hay que satisfacer. Luego, esa solución fraccionaria es cortada por esta restricción redundante.

Restricciones de esta forma se conocen en Programación Entera como “triggers”. El uso de este tipo de restricciones puede contribuir fuertemente a fortalecer formulaciones.

4.8 Consideraciones para problemas enteros-mixtos

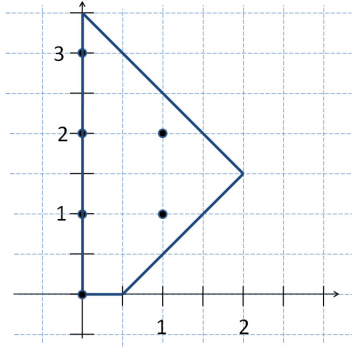
Consideraremos ahora el problema entero mixto:

$$\begin{aligned} \min \quad & c^T x \\ \text{s.a.} \quad & Ax \leq b \\ & x_j \text{ entero}, j \in J \end{aligned}$$

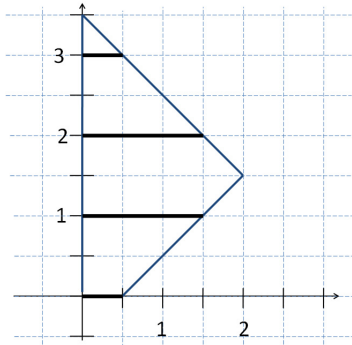
donde $J \subset \{1, \dots, n\}$. Podría parecer que toda la discusión anterior debiera aplicarse a la “parte entera” del problema y con eso estaríamos listos, pero existen varias complicaciones. Consideremos, por ejemplo, el siguiente problema:

$$\begin{aligned} \max \quad & x_1 \\ \text{s.a.} \quad & 2x_1 + 2x_2 \leq 7 \\ & 2x_1 - 2x_2 \leq 1 \\ & x_1 \geq 0, x_2 \geq 0, x_2 \text{ entero} \end{aligned}$$

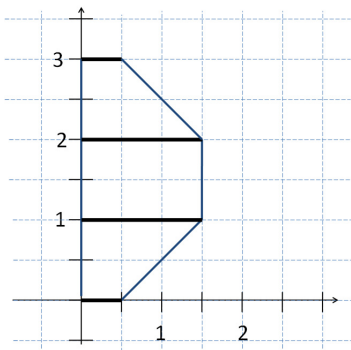
La figura muestra los puntos de coordenadas enteras que cumplen las restricciones:



Observemos, sin embargo, que x_1 no tiene que ser entero. La figura siguiente muestra (en líneas gruesas) los verdaderos puntos factibles para el problema:



La formulación ideal es diferente a la del problema entero puro, como se ve en la siguiente figura:



Como podemos ver, hay una mezcla de puntos con coordenadas enteras y continuas, y esto puede traer algunas complicaciones. Recordemos nuestro primer ejemplo de construcción de corte “por redondeo”:

$$\frac{13}{11}x_1 + \frac{20}{11}x_2 + x_3 + \frac{6}{11}x_4 \geq 6\frac{6}{11}$$

el cual era transformado a:

$$2x_1 + 2x_2 + x_3 + x_4 \geq 6\frac{6}{11}$$

Supongamos que *no* se requiere que x_2 y x_3 sean enteros. Entonces ya no podemos “apretar” el lado derecho a 7, ya que podría tomar valor fraccionario, y el punto que usamos como ejemplo en la sección anterior ya no sirve. Veremos ahora algunos tipos de cortes que toman en cuenta estas características.

Cortes Mixtos

Notación: Sea u un número.

- $\lfloor u \rfloor$ es el mayor entero $\leq u$ ($\lfloor 3.4 \rfloor = 3$)
- $\lceil u \rceil$ es el menor entero $\geq u$ ($\lceil 3.4 \rceil = 4$)
- $f(u) = u - \lfloor u \rfloor$ es la parte fraccionaria de u

Proposición: Consideremos el conjunto:

$$P = \{(x, y) : x + y \leq b, (x, y) \geq 0, y \in \mathbb{Z}\}$$

entonces la desigualdad:

$$\frac{x}{f(b)} + y \geq \lceil b \rceil$$

es válida para P .

Demostración: Consideremos dos casos respecto a y :

- $y \geq \lceil b \rceil$
- $y < \lceil b \rceil$ que equivale a $y \leq \lceil b \rceil - 1$

Cuando $y \geq \lceil b \rceil$ tenemos que

$$(\lceil b \rceil - y) \leq 0, -x \leq 0 \Rightarrow f(\lceil b \rceil - y) \leq 0, -x \leq 0$$

Sumando tenemos:

$$f(\lceil b \rceil - y) - x \leq 0$$

que es lo mismo que:

$$\frac{x}{f(b)} + y \geq \lceil b \rceil$$

Cuando $y < \lceil b \rceil$ tenemos que

$$y < \lceil b \rceil \Leftrightarrow y \leq \lceil b \rceil - 1 \Leftrightarrow (\lceil b \rceil - y) \geq 1$$

Luego:

$$(1 - f)(\lceil b \rceil - y) \geq 1 - f$$

Sumando a $x + y \geq b$ tenemos:

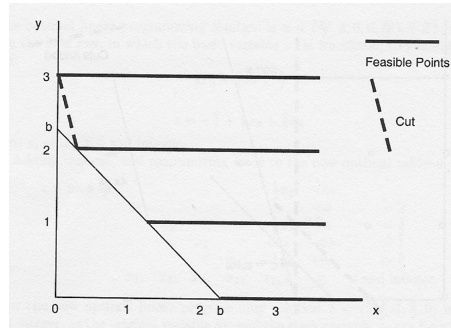
$$(1 - f)(\lceil b \rceil - y) + x + y \geq 1 - f + b$$

Desarrollando queda:

$$x \geq f(\lceil b \rceil - y) + 1 - f + b - \lceil b \rceil = f(\lceil b \rceil - y)$$

que es lo que queríamos obtener.

Gráficamente así se ve el corte:



(figura tomada de Wolsey, “Integer Programming”)

La idea anterior se puede generalizar, para esto consideremos ahora el conjunto:

$$P^{MIR} = \{(x, y) \in \mathbb{R}_+ \times \mathbb{Z}_+^2 : a_1 y_1 + a_2 y_2 \leq b + x\}$$

donde a_1, a_2, b son escalares, $b \notin \mathbb{Z}$.

Proposición: Sea $f = b - \lfloor b \rfloor$ y $g_i = a_i - \lfloor a_i \rfloor$, para $i = 1, 2$. Supongamos que $g_1 \leq f \leq g_2$. Entonces la desigualdad

$$\lfloor a_1 \rfloor y_1 + \left(\lfloor a_2 \rfloor + \frac{g_2 - f}{1 - f} \right) y_2 \leq \lfloor b \rfloor + \frac{x}{1 - f}$$

es válida para P^{MIR} .

Esta desigualdad se conoce como “Mixed Integer Rounding” (Wolsey)

Cortes Mixtos (o fraccionales) de Gomory

También existe una generalización del Corte de Gomory. Para esto supongamos que las variables del problema son (y, x) , con $y \in \mathbb{Z}_+^r$, y $x \geq 0$ pero continua. Al igual que en el caso entero puro, supongamos que hemos resuelto la relajación lineal y obtenido una solución (y^0, x^0) . Supongamos que hay una componente k tal que y_k^0 no es entero. Entonces, la fila k del tableau tiene la siguiente forma:

$$y_k + \sum_{j \in N_1} \bar{a}_{kj} y_j + \sum_{j \in N_2} \bar{a}_{kj} x_j = \bar{b}_k$$

donde N_1 son los índice de las variables y que son no básicas y N_2 es el respectivo para las variables x .

Proposición: Si $\bar{b}_k \notin \mathbb{Z}$, sea $f_j = \bar{a}_{kj} - \lfloor \bar{a}_{kj} \rfloor$, para $j \in N_1 \cup N_2$ y $f_0 = \bar{b}_k - \lfloor \bar{b}_k \rfloor$. Entonces, el siguiente Corte Mixto de Gomory es una desigualdad válida:

$$\sum_{j: f_j < f_0} f_j y_j + \sum_{j: f_j > f_0} \frac{f_0(1-f_j)}{1-f_0} y_j + \sum_{j: \bar{a}_{kj} > 0} \bar{a}_{kj} x_j + \sum_{j: \bar{a}_{kj} < 0} \frac{f_0}{1-f_0} \bar{a}_{kj} x_j \geq f_0$$

Al igual que con el corte entero puro, este corte se obtiene directamente del tableau, aunque el desarrollo de la construcción es bastante más complicado.

Desigualdades de “Flow Cover”

Consideremos n variables x y n variables y . Sean $N_1, N_2 \subset \{1, \dots, n\}$. Consideremos el conjunto definido por las siguientes desigualdades y condiciones:

$$\begin{aligned} \sum_{j \in N_1} x_j &\leq b + \sum_{j \in N_2} x_j \\ x_j &\leq a_j y_j \quad j \in N_1 \cup N_2 \\ x_j &\geq 0, y_j \in \{0, 1\} \quad j = 1, \dots, n \end{aligned}$$

Llamaremos P a este conjunto. ¿A qué tipo de problema se parecen las desigualdades anteriores?

Definición: Un conjunto $C = C_1 \cup C_2$ con $C_1 \subset N_1$, $C_2 \subset N_2$ se llama un “cover generalizado” para P si

$$\sum_{j \in C_1} a_j - \sum_{j \in C_2} a_j = b + \lambda, \quad \lambda > 0$$

(el valor λ se llama “exceso”).

Proposición: Sea $L_2 \subset N_2 - N_1$. La desigualdad de “flow cover”:

$$\sum_{j \in C_1} x_j + \sum_{j \in C_1} (a_j - \lambda)^+ (1 - y_j) \leq b + \sum_{j \in C_2} a_j + \lambda \sum_{j \in L_2} y_j + \sum_{j \in N_2 - (C_2 \cup L_2)} x_j$$

es válida para P .

El problema de separación para estas desigualdades es difícil de resolver en forma exacta pero es posible usar métodos heurísticos. También es necesario poder identificar la estructura adecuada para el “Flow Cover”.

4.9 Algoritmos de Branch and Cut

Un algoritmo de planos cortantes agrega restricciones en forma iterativa. Finalmente o se alcanza una solución entera o nos detenemos y se usa el Branch and Bound sobre el problema con todos los cortes para “redondear” la solución a enteros. Sin embargo, puede ser mejor idea ejecutar el Branch and Bound y simultáneamente ir agregando cortes.

Consideremos, como siempre, resolver el problema:

$$\begin{aligned} \max \quad & c^T x \\ \text{s.a.} \quad & Ax \leq b \\ & x \geq 0, \text{ entero} \end{aligned}$$

Supongamos que hemos resuelto la relajación lineal. Sea x^0 solución relajada. Supongamos que x_k^0 es fraccionaria y hemos decidido ramificar en esa variable. Como sabemos, se generarán dos subproblemas:

$$\begin{aligned} P_1) \quad & z^1 = \min_{\text{s.a.}} c^T x \\ & Ax \leq b \\ & x \geq 0 \\ & \boxed{x_k \leq \lfloor x_k^0 \rfloor} \\ P_2) \quad & z^2 = \min_{\text{s.a.}} c^T x \\ & Ax \leq b \\ & x \geq 0 \\ & \boxed{x_k \geq \lfloor x_k^0 \rfloor + 1} \end{aligned}$$

Una solución óptima del problema entero se encuentra en uno de estos dos subproblemas.

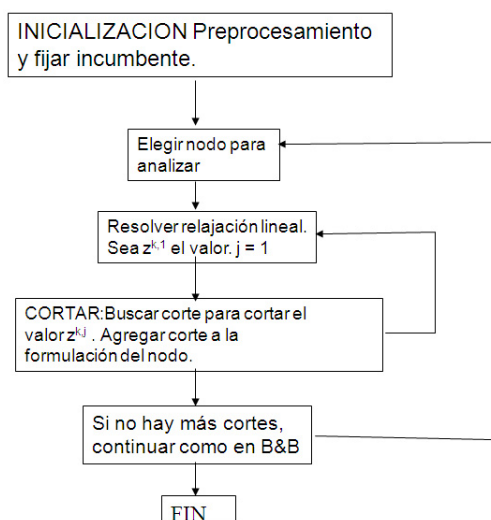
Consideremos entonces la siguiente idea: generar cortes para fortalecer cada uno de los dos subproblemas por separado. Esto puede ser efectivo ya que al fortalecer cada problema independientemente podemos aumentar las probabilidades de que se alcance una solución entera en alguno de ellos. Como sabemos, el disponer de soluciones enteras (aunque no sean necesariamente óptimas) es bueno ya que puede permitir la eliminación de muchas ramas del árbol.

La idea anterior, de agregar cortes en cada nodo del Branch and Bound, se conoce como “Ramificación y Corte” o “Branch and Cut”

Varias preguntas surgen de inmediato:

- ¿Qué cortes agregamos?
- ¿Por qué puede ser mejor hacer esto que agregar los cortes antes a la relajación lineal?
- ¿Cuál es el costo de hacer esto?

Esta es la idea general de un algoritmo de Branch and Cut:



4.10 Comentarios finales

- El algoritmo de Ramificación y Acotamiento fue desarrollado originalmente por Ailsa Land y Alison Doig en 1960, en el London School of Economics.
- Todos los software modernos de Branch and Bound son en realidad procedimientos de Branch and Cut. Ese es el caso del algoritmo para programación entera presente en Gurobi, por ejemplo.
- Suele haber una enorme ganancia en usar el enfoque de Branch& Cut debido a la eliminación masiva de ramas inútiles del árbol. Pero, por otro lado, puede existir una gran cantidad de familias de corte de donde elegir. Los sistemas de software tratan de hacer esto en forma automática, pero si el problema es muy difícil el usuario tendrá que intervenir.
- Los software, como Gurobi, generan cortes “generales”. Si queremos usar cortes especializados, tendremos que “intervenir” el software. (eso se puede hacer en Gurobi, por ejemplo)